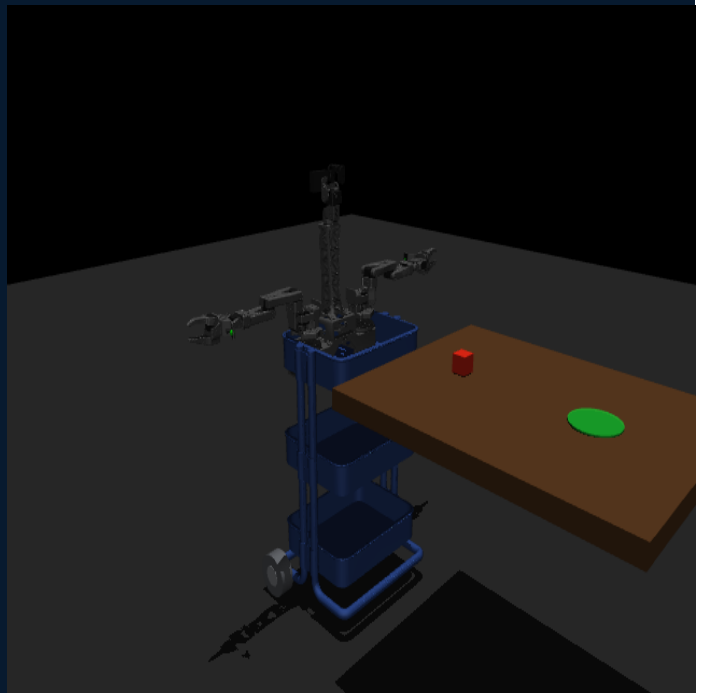


GEMELO DIGITAL

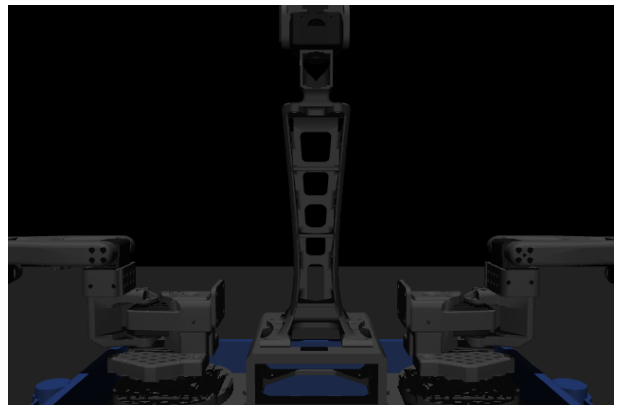
XLeRobot 0.4

Guía completa de puesta en marcha,
teleoperación, datasets y
entrenamiento

MUJOCO / ISAAC SIM / GAMEPAD / RL



Base 0.4: ruedas motrices exteriores



Cuello, cámara y manipuladores

ESTADO DEL PROYECTO

Modelo funcional para simulación. La geometría y la cadena cinemática están integradas; la fidelidad física absoluta exige completar las mediciones del robot real.

Contenido

- 1 Preparación y verificación
- 2 Inspección del gemelo 0.4
- 3 Teleoperación en MuJoCo
- 4 Importación y teleoperación en Isaac Sim
- 5 Datasets y formato LeRobot
- 6 Entrenamiento CEM y PPO
- 7 Casos de uso, calibración y sim-to-real
- 8 Diagnóstico y seguridad

Ruta rápida

Si quieres comprobar hoy mismo que toda la cadena funciona, sigue estas seis fases en orden.

- | | |
|-----------|---|
| 01 | Preparar
Ejecuta bootstrap.ps1 sólo si no existe .venv. |
| 02 | Verificar
Ejecuta verify.ps1. No entrenes si alguna prueba falla. |
| 03 | Ver
Genera las tres capturas y revisa ruedas, cámara y brazos. |
| 04 | Mover
Teleopera MuJoCo con A como hombre muerto y B como E-stop. |
| 05 | Repetir
Regenera el USD y ejecuta el smoke test de Isaac Sim. |
| 06 | Aprender
Crea datasets; después prueba CEM y PPO. |

REGLA DE ORO

Primero verifica y teleopera; después genera datos; sólo entonces entrena. El robot físico queda fuera de esta guía y no recibe órdenes.

1. Propósito del manual

Este documento explica qué contiene el proyecto, cómo verificarlo desde cero y cómo empezar a utilizarlo en MuJoCo e Isaac Sim. Todos los procedimientos son para simulación. No conectan ni envían órdenes al XLeRobot físico.

Ruta utilizada en este PC:

```
C:\Users\farqu\OneDrive\Documentos\ChatGPT\xlerobot
```

Abre PowerShell y sitúate siempre en esa carpeta:

```
cd "C:\Users\farqu\OneDrive\Documentos\ChatGPT\xlerobot"
```

2. Qué está construido

El gemelo representa la versión XLeRobot 0.4 de dos ruedas y contiene:

- carro y piezas CAD derivadas del diseño 0.4 oficial;
- dos ruedas motrices de 127 mm con colisión, fricción, cubo y neumático visible;
- dos apoyos pasivos interiores y base diferencial móvil;
- dos brazos SO-101, cada uno con cinco articulaciones y una pinza;
- cuello con pan/tilt y cámara orientada hacia el lado operativo (-x);
- 16 articulaciones controlables en total;
- cámaras simuladas neck_rgb y neck_depth;
- teleoperación con mando Xbox, Switch Pro o compatible;
- entorno Gymnasium, episodios simulados y entrenamiento CEM/PPO;
- URDF canónico e importación reproducible a Isaac Sim.

La misma estructura cinemática se genera para ambos simuladores. Las pruebas comprueban que nombres, límites, cámaras y ruedas no diverjan.

Qué todavía es provisional

El modelo es funcional, pero no es todavía un gemelo físico exacto. La vía de las ruedas es provisional (0,500 m) y faltan medidas reales de masas, inercias, centros de gravedad, fricción, neumáticos, holguras, latencia, respuesta de servos e intrínsecas/extrínsecas de cámara.

3. Preparación inicial en Windows

Si .venv ya existe, pasa a la sección 4. Para preparar un PC nuevo:

```
.\scripts\bootstrap.ps1
```

El equivalente manual es:

```
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
pip install -e .
```

Para PPO instala también:

```
.\.venv\Scripts\python.exe -m pip install -r requirements-rl.txt
```

4. Verificación completa

```
.\scripts\verify.ps1
```

El resultado correcto incluye:

```
PASS: XLeRobot 0.4.0 servo dual-wheel
PASS: canonical URDF
10 passed
Verification complete.
```

Se comprueban MJCF, 16 actuadores, ruedas, neumáticos visibles, CAD, URDF, cámara, movimiento diferencial, seguridad del mando y creación de un episodio. No continúes a entrenamiento si falla.

5. Inspección visual

```
.\.venv\Scripts\python.exe scripts\render_twin_snapshot.py
```

Se crean:

- outputs/xlerobot_v04_overview.png: vista general;
- outputs/xlerobot_v04_front.png: ruedas, cubos y soportes;
- outputs/xlerobot_v04_head.png: cuello, cámara y brazos.

En la vista frontal deben verse los neumáticos completos fuera del carro y no la antigua carcasa CAD central. La lente debe mirar hacia el lado operativo.

6. MuJoCo

6.1 Abrir y teleoperar

```
.\.venv\Scripts\python.exe scripts\run_mujoco.py
```

MuJoCo es la ruta recomendada para física, articulaciones, recompensas, datasets y primeras políticas: arranca rápido y no necesita regenerar USD.

Para ver desde la cámara del cuello:

```
.\.venv\Scripts\python.exe scripts\run_mujoco.py --camera neck_rgb
```

6.2 Mapa del mando

Control	Sin bumper	Con LB	Con RB
Mantener A	Habilita movimiento	Habilita movimiento	Habilita movimiento
Stick izquierdo	Avance y giro	Rotación/pitch izquierdo	Rotación/pitch derecho
Stick derecho	Pan/tilt del cuello	Codo/muñeca izquierda	Codo/muñeca derecha
D-pad izquierda/derecha	—	Roll de muñeca	Roll de muñeca
Gatillos LT/RT	—	Abrir/cerrar pinza	Abrir/cerrar pinza
LB + RB	—	Control bimanual	Control bimanual

Control	Sin bumper	Con LB	Con RB
B	E-stop enclavado	E-stop enclavado	E-stop enclavado
A + Start	Libera E-stop y home	Igual	Igual

A es el hombre muerto. Al soltarlo la base se detiene. B permanece enclavado hasta A + Start.

6.3 Prueba de aceptación

1. Sin A, los sticks no deben mover nada.
2. Con A, avanzar debe girar ambas ruedas igual.
3. Girar debe mover las ruedas en sentidos opuestos.
4. Al soltar A, la base debe detenerse.
5. Mueve el cuello y observa el cambio en neck_rgb.
6. Prueba cada brazo con LB y RB, y ambas pinzas con los gatillos.
7. Pulsa B: todo debe quedar bloqueado.
8. Pulsa A + Start: se libera el E-stop y vuelve a home.

7. Isaac Sim

7.1 Cómo usarlo

Isaac es apropiado para cámaras RTX, escenas visualmente ricas, sensores, generación sintética y futura paralelización en Isaac Lab. MuJoCo es más simple para depuración y primeras políticas.

La instalación validada en este PC está en C:\isaacsim.

7.2 Regenerar el USD

```
.\isaac\import.ps1 -IsaacRoot "C:\isaacsim"
```

El script importa el URDF, mantiene la base flotante, crea una nueva revisión, añade neck_rgb/neck_depth y valida 16 joints. La revisión actualmente validada es:

```
isaac/generated/xlerobot_v04_4/xlerobot_v04.usda
```

7.3 Abrir para inspección

```
.\isaac\open.ps1 -IsaacRoot "C:\isaacsim"
```

Selecciona automáticamente el USD más reciente.

7.4 Smoke test sin ventana

```
C:\isaacsim\python.bat .\isaac\teleop_gamepad.py --headless --smoke-steps 10
```

Debe imprimir Validated articulation: 16 joints y finalizar con código cero. Isaac 6.0.1 puede imprimir errores Jinja2 de una extensión opcional de ruedas; nuestro lanzador no depende de ella.

7.5 Teleoperar

```
C:\isaacsim\python.bat .\isaac\teleop_gamepad.py
```

El script añade suelo físico y utiliza el mismo mapa y seguridad que MuJoCo. Comprueba primero base/E-stop y después cuello, brazos y pinzas.

8. Datasets simulados

8.1 Rollouts aleatorios para validar formato

```
.\.venv\Scripts\python.exe scripts\record_rollouts.py `
--episodes 10 --images `
--output datasets\raw\reach_random
```

Cada .npz contiene observaciones, acciones, recompensas, tarea e imágenes de neck_rgb. Sirve para validar el pipeline, no para imitación competente.

8.2 Demostraciones del experto IK

```
.\.venv\Scripts\python.exe scripts\record_expert.py `
--episodes 100 --images --max-attempts 500 `
--output datasets\expert\reach
```

Los intentos fallidos se descartan. Revisa cantidad, recompensas e imágenes.

8.3 Exportar a LeRobotDataset v3

```
.\.venv\Scripts\python.exe -m pip install -r requirements-lerobot.txt
.\.venv\Scripts\python.exe scripts\export_lerobot.py datasets\expert\reach
```

No mezcles datasets de orientaciones antiguas con la cámara actual. Conserva el commit Git usado para generar cada dataset.

9. Entrenamiento

9.1 CEM: prueba mínima sin GPU

```
.\.venv\Scripts\python.exe scripts\train_cem.py `
--iterations 40 --population 32 `
--output outputs\models\cem_reach.npz

.\.venv\Scripts\python.exe scripts\evaluate_cem.py `
outputs\models\cem_reach.npz --episodes 10
```

CEM comprueba que observaciones, acciones, recompensa y checkpoints están conectados; no garantiza una política competente.

9.2 PPO de brazo derecho

```
.\.venv\Scripts\python.exe scripts\train_r1.py `
--action-mode right_arm --timesteps 100000 `
--output outputs\models\ppo_reach

.\.venv\Scripts\python.exe scripts\evaluate_r1.py `
outputs\models\ppo_reach.zip `
--action-mode right_arm --episodes 10
```

Alternativa de un solo comando:

```
.\scripts\train.ps1 -Timesteps 100000 -Output outputs\models\ppo_reach
```

La tarea inicial mueve cuatro articulaciones del brazo derecho para alcanzar un cubo, con la base quieta. Es una prueba previa a locomoción o agarre.

9.3 Modelos externos

`models/models.yaml` cataloga SmolVLA, checkpoints pi0.5 y una política ACT de agarre. Estar catalogado no implica compatibilidad directa: hay que comprobar tarea, orden de joints, normalización, cámaras y dimensiones.

Ejemplo de descarga explícita:

```
.\.venv\Scripts\python.exe scripts\download_model.py lerobot/smolvla_base
```

Nunca envíes directamente las acciones de un checkpoint externo al robot real.

10. Casos de uso para comenzar

10.1 Conducción manual

Empieza en MuJoCo y repite en Isaac. Recorre un circuito simple y registra distancia, ángulo y tiempo. Permite ajustar radio efectivo, vía y fricción.

10.2 Inspección con la cámara del cuello

Conduce observando sólo `neck_rgb` y usa `pan/tilt` para mantener un objeto centrado. Puede evolucionar hacia seguimiento visual o navegación asistida.

10.3 Alcance con un brazo

Entrena `right_arm` para alcanzar un cubo. Valida límites, cinemática, recompensa y RL antes de introducir base móvil y pinza.

10.4 Agarre y colocación

Itera en MuJoCo y usa Isaac para percepción RTX. Requiere calibrar contacto, fricción y cierre de pinza. Entrena con demostraciones competentes.

10.5 Dataset visual para imitación o VLA

MuJoCo permite volumen rápido; Isaac, variación visual más rica. Guarda imagen, estado, acción e instrucción por frame. Aleatoriza objetos, luz, texturas y pose.

10.6 Manipulación bimanual

Usa `LB + RB` para coordinar brazos y grabar demostraciones. Comienza con objetos grandes, velocidades bajas y una métrica clara de coordinación.

10.7 VR

Isaac será la ruta de renderizado. Ya existen cámara y joints, pero siguen pendientes el adaptador OpenXR, IK desde controladores, vista estereoscópica y prueba con visor real.

10.8 Sim-to-real

Entrena con randomización, evalúa en escenas no vistas y transfiere con límites de velocidad. No debe comenzar hasta completar la calibración física.

11. Mediciones pendientes

Registra en `calibration/measurements.yaml`:

- vía y posición exacta del eje;
- diámetro/ancho de neumáticos bajo carga;
- masas, inercias y centros de gravedad;
- ceros y límites mecánicos;
- par, velocidad, damping, fricción, backlash y respuesta a escalón;
- fricción rueda-suelo y apoyos pasivos;
- FOV, f_x , f_y , c_x , c_y , distorsión y exposición;
- transformadas base- $\rightarrow$ neck, neck- $\rightarrow$ camera, wrist- $\rightarrow$ gripper;
- latencia de cámara, red y actuadores;
- rectas, giros completos y agarres repetidos.

Después hay que regenerar USD, repetir pruebas y volver a crear cualquier dataset o política dependiente de cámara/física.

12. Orden recomendado

1. Ejecutar `verify.ps1`.
2. Inspeccionar las tres imágenes.
3. Teleoperar MuJoCo y probar E-stop.
4. Verificar `neck_rgb`.
5. Regenerar y probar Isaac headless.
6. Teleoperar Isaac.
7. Generar diez episodios con imágenes.
8. Generar demostraciones expertas.
9. Ejecutar CEM y PPO de brazo derecho.
10. Elegir un caso de uso y una métrica de éxito.
11. Medir/calibrar el robot real.
12. Preparar después sim-to-real y VR.

13. Diagnóstico rápido

Problema	Comprobación
MuJoCo no abre	Ejecutar <code>verify.ps1</code> y revisar <code>.venv</code>
No detecta el mando	Probar USB, verificarlo en Windows y reiniciar
No se mueve	Mantener A; liberar E-stop con A + Start
Cámara incorrecta	Regenerar; no abrir un USD antiguo
Isaac muestra geometría antigua	Ejecutar <code>import.ps1</code> ; abrir la revisión nueva

Problema	Comprobación
Isaac imprime errores Jinja2	Confirmar 16 joints y código de salida cero
La política no aprende	Revisar recompensa, acción, semillas y evaluación
Dataset sin imágenes	Añadir <code>--images</code>

14. Seguridad antes del robot real

- E-stop físico accesible y probado;
- velocidad y par inicialmente limitados;
- robot elevado para la primera prueba de ruedas;
- una sola articulación en la primera prueba de brazos;
- zona despejada;
- watchdog de comunicación;
- signo, cero y límites de cada joint comprobados;
- política validada primero en simulación y luego incrementalmente.

RESULTADO ESPERADO

Al completar la ruta tendrás el mismo robot articulado en MuJoCo e Isaac Sim, teleoperación segura con gamepad, captura de observaciones y acciones, y dos rutas de entrenamiento reproducibles.